

Evaluación Continua 3

Se cuenta con un *scanner*, *parser* y *visitor* que trabajan sobre la siguiente gramática:

```
Program ::= StmtList
StmtList ::= Stmt { ; Stmt } *
Stmt ::= Id = CExp | print ( CExp )
CExp ::= Expr { ( + | - ) Expr } *
Expr ::= Term { ( * | / ) Term } *
Term ::= Factor [ * Factor ]
Factor ::= Number | ( CExp ) | sqrt( CExp ) | Id
```

Esta gramática define un mini-lenguaje imperativo que permite construir programas compuestos por una lista de sentencias separadas por punto y coma. Dichas sentencias pueden ser asignaciones de valores a variables mediante expresiones aritméticas o instrucciones de salida con **print**. Ejemplos de programas válidos:

1. `x = 5; y = 10; print(x + y)`
2. `a = 2 ** 3; b = sqrt(16); print(a * b)`
3. `m = (3 + 7) * 2; n = m - 5; print(n)`
4. `u = 100 / 4; v = u + 6; w = v ** 2; print(w)`
5. `x = sqrt(25); y = (x + 3) * 2; print(y - 1)`
6. `a = 7; b = 8; c = (a + b) / 3; print(c)`
7. `r = 2; s = r ** 4; t = sqrt(s); print(t)`
8. `p = 9; q = p - 4; r = q * (2 + 3); print(r)`

La gramática debe ser extendida de modo que el *scanner*, el AST, el *parser* y el *visitor* soporten las siguientes construcciones adicionales:

```
Program ::= StmtList
StmtList ::= Stmt { ; Stmt } *
Stmt ::= Id = CExp | print ( CExp )
       ::= Id { , Id } * = Cexp { , Cexp } *
CExp ::= Expr { ( + | - ) Expr } *
Expr ::= Term { ( * | / ) Term } *
Term ::= Factor [ * Factor ]
Factor ::= Number | ( CExp ) | sqrt( CExp ) | Id
        ::= max ( Cexp { , Cexp } * ) | -( CExp )
        ::= ifexp ( Cexp , Cexp , Cexp )
```

Ejemplos de cadenas válidas con las nuevas reglas:

1. `x = 5; print(x)`
2. `x = 5; y = -(8); print(x+y)`
3. `x , y = 4,5 ; print( x+y)`
4. `m = max(7,3,10); print(m)`
5. `x = 10; y = 4; m = man(x,y+5,20); print(m)`
6. `x = 8; y = 4; u, v = x+y, x-y; print(u+v)`
7. `p = max(3,7,2,9,4); q = sqrt((p+6)*(p+6)); print(p+q)`
8. `x, y, z = 2,3,4; a = if(x,y,z); print(z*z + a)`
9. `m = -(7+5); n = 2; r1, r2 = m*n, m/n; print( max(r1,r2,m+n))`
10. `a = 4; b = 9; c = 16; x, y = sqrt(a)+sqrt(b), max(a,b,c); print(x+y)`

Pregunta

Explique los cambios que deben incorporarse en el scanner, el parser, el AST y el visitor para soportar - CExp en el lenguaje. Describa al menos diez pasos claramente enumerados que indiquen las modificaciones necesarias en cada uno de estos componentes del compilador.